МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Реализация метода классификации данных с использованием природных временных рядов.

Студент гр. 3384 _____ Рудаков А.Л.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург
2026

Цель работы.

Построение классификатора данных геофизического мониторинга на примере наблюдений нейтронных мониторов.

Задание.

Вариант 20(3). Якутск (YKTK) 1-14 февраля 2026

1. Подготовить данные. Скачать данные из источника [https://www.nmdb.eu/nest/] за период и станцию, указанные в варианте. Использовать данные с одноминутным временным разрешением. При наличии пропусков в данных заполнить их медианным значением (функция median в MatLab).

2. Сформировать векторы-истории. Временной ряд разбить на последовательные блоки (векторы) длиной m, каждый из которых должен содержать m прошлых значений. Последний вектор в ряду будет «тестовой» точкой для поиска соседей. Величину m выбрать равной 180 отсчётов.

3. Найти ближайших соседей. Для «тестового» вектора вычислить расстояние (метрику) до всех остальных векторов-историй (использовать 3 различные метрики). Число соседей k определить равным 2–4.

4. Использовать соседей для прогноза. На основе найденных ближайших соседей построить предсказания для будущих значений ряда. При нахождении нескольких схожих отрезков взять простое среднее следующих за ними значений.

5. Сделать соответствующие выводы, оформить отчёт по работе.

Выполнение работы.

Работа выполнялась в Jupiter Notebook, Python.

Для начала были найдены пропуски данных во временном ряду, путем сравнения полученных данных с эталонным временным рядом с шагом в 1 минуту и заполнены медианным значением.

Далее временной ряд был разделен на последовательные векторы-истории длиной m = 180 отсчетов с шагом в 1 минуту (скользящее окно). Таким образом, было сформировано 19980 векторов-историй. Последние 180 значений ряда были выделены в отдельный «тестовый» вектор, для которого требовалось построить прогноз.

Далее для «тестового» вектора было вычислено расстояние до всех остальных векторов-историй с использованием трех различных метрик: евклидова расстояния, манхэттенского расстояния и расстояния Чебышева. Количество ближайших соседей (k) было выбрано равным 3. В результате для каждой метрики были найдены свои три ближайших соседа, индексы и значения расстояний до которых были зафиксированы.

После чего на основе найденных ближайших соседей был построен прогноз будущего значения ряда. Для этого для каждого из трех соседей бралось значение, следующее непосредственно за вектором-историей, после чего вычислялось их простое среднее. Таким образом, были получены три независимых прогноза (по одному на каждую метрику), а также рассчитан итоговый ансамблевый прогноз как среднее трех полученных значений.

Графики сравнения тестового вектора с найденными соседями для каждой из трех метрик показаны на рис.1, рис.2, рис.3. Итоговый график прогноза, наложенный на оригинальный временной ряд, представлен на рис. 4.

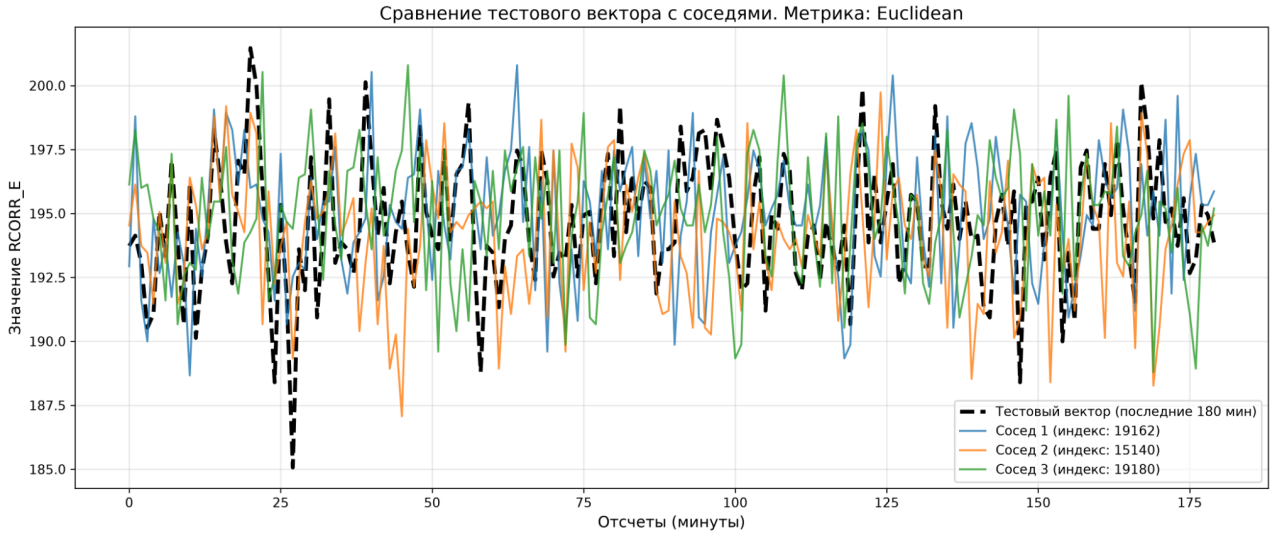


Рисунок 1 - график сравнения с соседями по метрике Euclidean

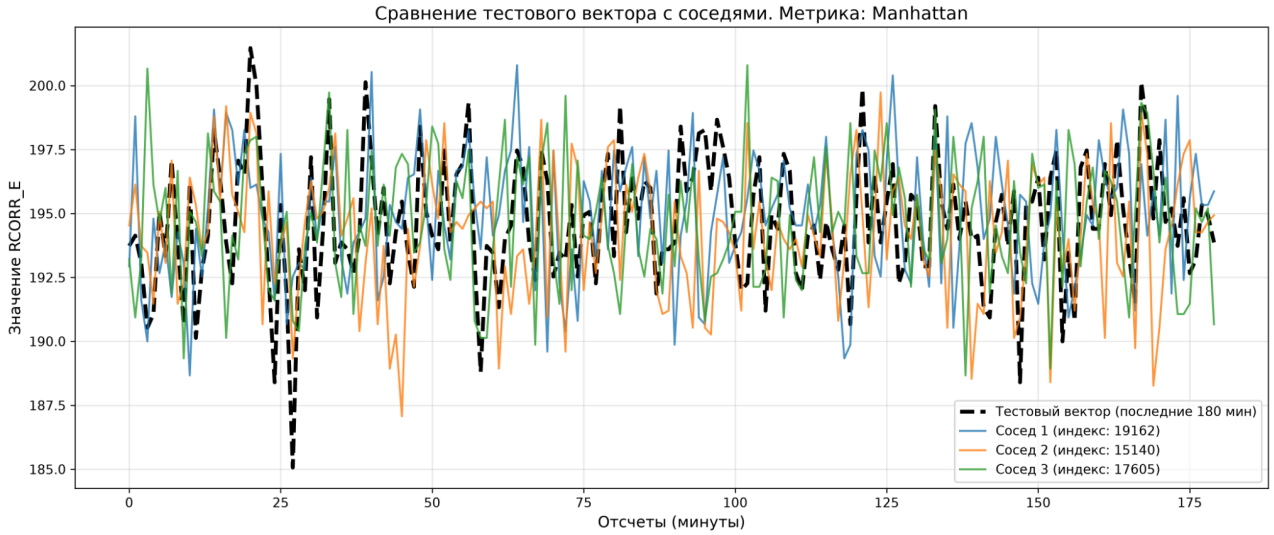


Рисунок 2 - график сравнения с соседями по метрике Manhattan

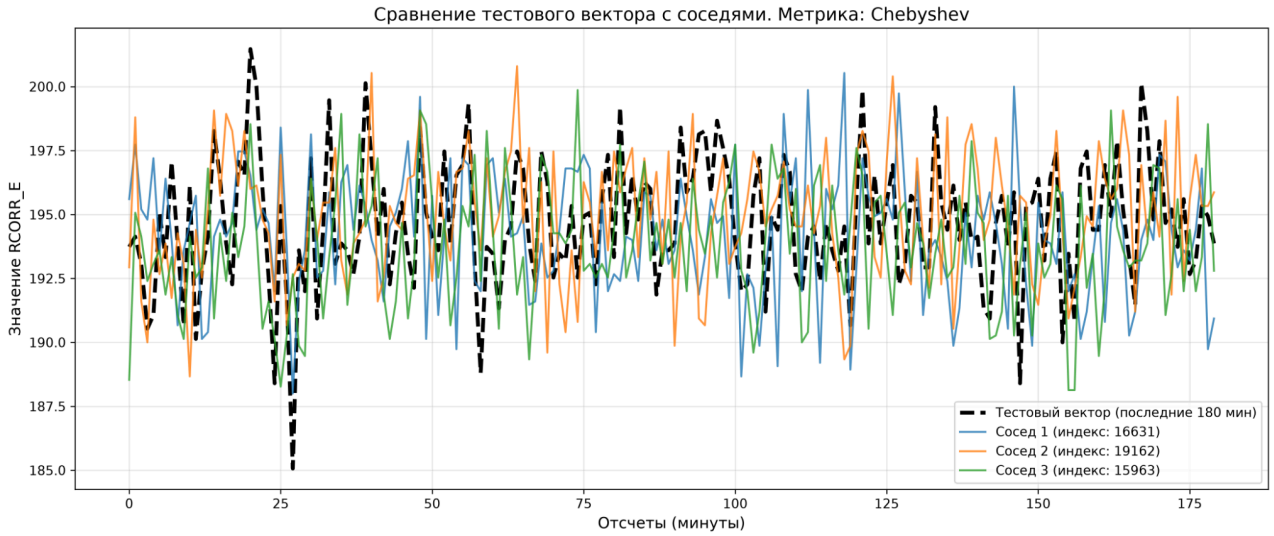


Рисунок 3 - график сравнения с соседями по метрике Chebyshev

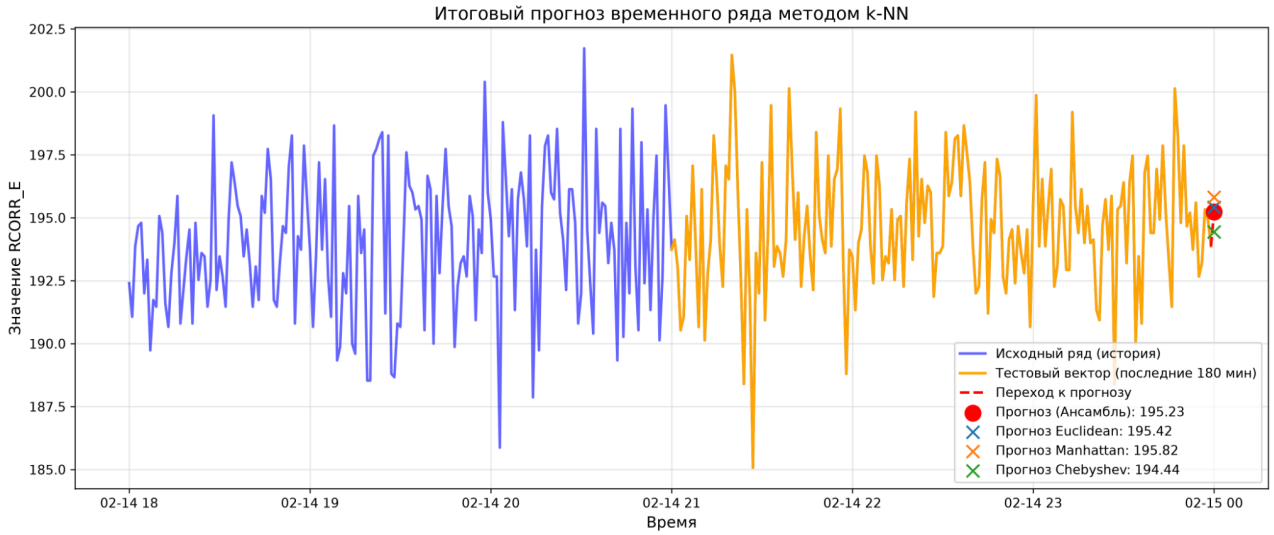


Рисунок 4 - график прогноза

Выводы.

В ходе лабораторной работы было выполнено прогнозирование временного ряда нейтронного монитора методом k-ближайших соседей (k-NN). Исходный ряд был разбит на 19980 векторов-историй длиной 180 отсчетов. Для «тестового» вектора были найдены по 3 ближайших соседа с использованием трех метрик расстояния: евклидовой, манхэттенской и Чебышева. Прогноз осуществлялся путем усреднения значений, следующих за найденными историческими фрагментами. Итоговый ансамблевый прогноз был получен как среднее прогнозов по всем трем метрикам. Приведены графики сравнения соседей и итогового прогноза.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД

Файл main.ipynb:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import pairwise_distances

with open('clear_data.txt', 'w') as f:
    f.write('start_date_time RCORR_E\n')

df_sigma.to_csv(
    'clear_data.txt',
    sep=';',
    index=False,
    header=False,
    mode='a',
    date_format='%Y-%m-%d %H:%M:%S',
    float_format='%.3f'
)

df = pd.read_csv(
    'clear_data.txt',
    sep=';',
    skiprows=1,
    names=['start_date_time', 'RCORR_E'],
    parse_dates=['start_date_time']
)

series = df['RCORR_E'].values
print(df.head())

m = 180
k = 3

if len(series) <= m:
    raise ValueError(f"Длина ряда ({len(series)}) должна быть больше m ({m}).")

history_vectors = []
targets = []

for i in range(len(series) - m):
    history_vectors.append(series[i : i + m])
    targets.append(series[i + m])

history_vectors = np.array(history_vectors)
targets = np.array(targets)
test_vector = series[-m:].reshape(1, -1)

print(f"\nСформировано векторов-историй: {len(history_vectors)}")
```

```
metrics = ['euclidean', 'manhattan', 'chebyshev']
neighbors_dict = {}

print("\n--- Поиск ближайших соседей ---")
for metric in metrics:
    distances = pairwise_distances(test_vector, history_vectors, metric=metric).flatten()

    nearest_indices = np.argsort(distances)[:k]

    neighbors_dict[metric] = {
        'indices': nearest_indices,
        'distances': distances[nearest_indices]
    }

    print(f"\nМетрика: {metric}")
    print(f"  Индексы соседей: {nearest_indices}")
    print(f"  Расстояния: {distances[nearest_indices]}")

predictions = {}

print("\n--- Прогнозирование ---")
for metric in metrics:
    neighbor_indices = neighbors_dict[metric]['indices']
    neighbor_targets = targets[neighbor_indices]

    # Прогноз как простое среднее следующих значений
    prediction = np.mean(neighbor_targets)
    predictions[metric] = prediction

    print(f"Метрика: {metric}")
    print(f"  Значения следующих точек у соседей: {neighbor_targets}")
    print(f"  Прогноз (среднее): {prediction:.4f}")

ensemble_prediction = np.mean(list(predictions.values()))
print(f"\nАнсамблевый прогноз (среднее по 3 метрикам): {ensemble_prediction:.4f}")


import os

output_dir = 'plots'
os.makedirs(output_dir, exist_ok=True)

for metric in metrics:
    plt.figure(figsize=(14, 6))

    plt.plot(test_vector.flatten(), label='Тестовый вектор (последние 180 мин)',
             color='black', linewidth=3, linestyle='--')

    indices = neighbors_dict[metric]['indices']

    for j, idx in enumerate(indices):
        plt.plot(history_vectors[idx], label=f'Сосед {j+1} (индекс: {idx})',
                 alpha=0.8, linewidth=1.5)
```

```
            plt.title(f'Сравнение тестового вектора с соседями. Метрика: {metric.capitalize()}',
fontsize=14)
            plt.xlabel('Отсчеты (минуты)', fontsize=12)
            plt.ylabel('Значение RCORR_E', fontsize=12)
            plt.legend(fontsize=10)
            plt.grid(True, alpha=0.3)
            plt.tight_layout()

            filename = os.path.join(output_dir, f'neighbors_{metric}.png')
            plt.savefig(filename, dpi=300)
            print(f"Сохранен график: {filename}")
            plt.show()
            plt.close()

        plt.figure(figsize=(14, 6))

        plot_window = m * 2
        if len(df) < plot_window:
            plot_window = len(df)

        historical_data = df.iloc[-plot_window:]
        plt.plot(historical_data['start_date_time'], historical_data['RCORR_E'],
                 label='Исходный ряд (история)', color='blue', alpha=0.6, linewidth=2)

        test_data = df.iloc[-m:]
        plt.plot(test_data['start_date_time'], test_data['RCORR_E'],
                 label='Тестовый вектор (последние 180 мин)', color='orange', linewidth=2)

        last_time = df['start_date_time'].iloc[-1]
        next_time = last_time + pd.Timedelta(minutes=1) # Следующая минута
        last_value = df['RCORR_E'].iloc[-1]

        plt.plot([last_time, next_time],
                 [last_value, ensemble_prediction],
                 color='red', linestyle='--', linewidth=2, label='Переход к прогнозу')

        plt.scatter([next_time], [ensemble_prediction], color='red', s=150, zorder=5,
                    label=f'Прогноз (Ансамбль): {ensemble_prediction:.2f}')

        for metric, pred in predictions.items():
            plt.scatter([next_time], [pred], marker='x', s=100, zorder=6,
                        label=f'Прогноз {metric.capitalize()}: {pred:.2f}')

        plt.title('Итоговый прогноз временного ряда методом k-NN', fontsize=14)
        plt.xlabel('Время', fontsize=12)
        plt.ylabel('Значение RCORR_E', fontsize=12)
        plt.legend(loc='best', fontsize=10)
        plt.grid(True, alpha=0.3)
        plt.tight_layout()

        filename = os.path.join(output_dir, 'final_forecast.png')
        plt.savefig(filename, dpi=300)
```

```
plt.show()
plt.close()
```